

Gestión Centralizada de Configuración y Ambientes

`veterinaria-config` (Spring Cloud Config Server) sirviendo configuración DEV/PROD a `veterinaria-mascota-ms` y al microservicio replicado `citas-ms`

Basado en la guía `S02_Configuracion_Centralizada_Ambientes - 262dist/pagatu`

Repositorio	Tarea-1 (monorepo: infra/ + services/)
Autor / commits	JoaHasss
Stack añadido	Spring Cloud Config 2025.1.2 (Oakwood)
Fecha del informe	27 de agosto de 2026
Sesión previa	S1 — <code>veterinaria-mascota-ms</code> (CRUD base)

1. Objetivo y alcance

Este informe documenta la Sesión 2 del curso, cuyo propósito es "construir e implementar un Config Server que centraliza la configuración de un microservicio por ambiente, verificando consultas HTTP y migrando el microservicio para leer configuración externa en DEV." El ejercicio guiado (sección 3 de la guía) se aplicó sobre `veterinaria-mascota-ms` (el servicio construido en S1), y la actividad autónoma (sección 4, "CREA") se resolvió replicando el mismo patrón sobre un **segundo microservicio nuevo**: `citas-ms` (citas veterinarias), en lugar del `orden-ms` genérico que usa la guía original.

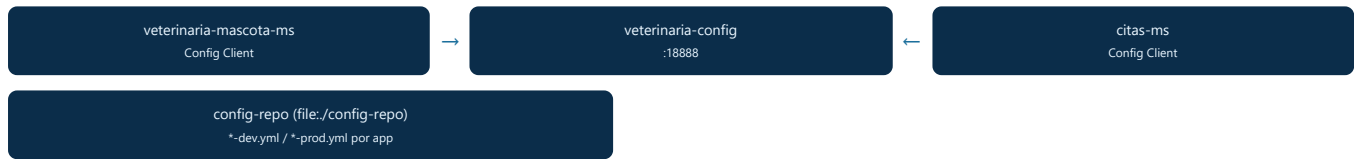
Cambio estructural importante. Hasta S1, este repositorio tenía el microservicio en la raíz. La guía S2 asume un monorepo con `services/` e `infra/` (igual que el repositorio de referencia `262dist/pagatu`). Se migró la estructura completa en este commit: `veterinaria-mascota-ms` se movió a `services/veterinaria-mascota-ms` , y se creó `infra/veterinaria-config` para el Config Server. Este cambio es necesario y se mantendrá en S3/S4, donde se añadirán Eureka y el API Gateway como nuevos componentes de `infra/` .

1.1 Qué se construyó

Componente	Ubicación	Rol
<code> veterinaria-config </code>	<code> infra/veterinaria-config </code>	Config Server (Spring Cloud Config), puerto 18888, repositorio nativo local
<code> config-repo </code>	<code> infra/veterinaria-config/config-repo </code>	4 archivos YAML versionados: dev/prod de cada microservicio
<code> veterinaria-mascota-ms </code>	<code> services/veterinaria-mascota-ms </code>	Migrado de configuración local a Config Client (S1 → S2)
<code> citas-ms </code>	<code> services/citas-ms </code>	Microservicio nuevo, construido ya como Config Client desde el inicio (actividad autónoma)

2. Arquitectura de configuración centralizada

El patrón implementado corresponde a **Centralized Configuration** (categoría Gobernanza). En vez de que cada microservicio cargue su propio `application-dev.yml` / `application-prod.yml` desde su classpath, todos consultan al mismo Config Server por HTTP al arrancar.



Convención de resolución: el Config Server forma la URL `/{{spring.application.name}}/{{perfil}}` y busca el archivo `{{spring.application.name}}-{{perfil}}.yml` dentro de `config-repo`. Por eso `spring.application.name` en cada microservicio debe coincidir exactamente con el prefijo del archivo.

2.1 Estructura final del monorepo

```
Tarea-1/
├─ infra/
│   └─ veterinaria-config/          (Config Server, puerto 18888)
│       └─ config-repo/
│           ├── veterinaria-mascota-ms-dev.yml
│           ├── veterinaria-mascota-ms-prod.yml
│           ├── citas-ms-dev.yml
│           └─ citas-ms-prod.yml
│       └─ src/main/java/pe/edu/upeu/config/VeterinariaConfigApplication.java
└─ services/
    ├── veterinaria-mascota-ms/      (S1, migrado a Config Client)
    │   └─ src/main/resources/application.yml (minimalista)
    └─ citas-ms/                    (nuevo, S2 - actividad autónoma)
        ├── src/main/java/pe/edu/upeu/citas/... (mismo patrón arquitectónico de S1)
        └─ src/main/resources/db/migration/V1_create_citas_tables.sql
```

3. Implementación del Config Server

3.1 Dependencias y BOM (`infra/veterinaria-config/pom.xml`)

```
<properties>
  <java.version>21</java.version>
  <spring-cloud.version>2025.1.2</spring-cloud.version>
</properties>

<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-config-server</artifactId>
</dependency>

<!-- BOM: fija todas las versiones del ecosistema Spring Cloud 2025.1.2 (Oakwood),
      compatible con Spring Boot 4.0.7 -->
<dependencyManagement><dependencies><dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-dependencies</artifactId>
  <version>${spring-cloud.version}</version>
  <type>pom</type> <scope>import</scope>
</dependency></dependencies></dependencyManagement>
```

3.2 Clase principal

```
@SpringBootApplication
@EnableConfigServer
public class VeterinariaConfigApplication {
    public static void main(String[] args) {
        SpringApplication.run(VeterinariaConfigApplication.class, args);
    }
}
```

3.3 `application.yml` — repositorio nativo

```
server:
  port: 18888
spring:
  application:
    name: veterinaria-config
  profiles:
    active: native
  cloud:
    config:
      server:
        native:
          search-locations: ${CONFIG_REPO_LOCATION:file:./config-repo}
```

En DEV no se define `CONFIG_REPO_LOCATION`, así que usa el valor por defecto `file:./config-repo`, relativo al directorio donde corre el proceso Maven (`infra/veterinaria-config`, no la raíz del repo).

4. Configuración externa por ambiente (`config-repo`)

Se crearon 4 archivos, uno por combinación aplicación/perfil. A continuación, el contraste dev/prod para `veterinaria-mascota-ms` (el mismo patrón se replicó para `citas-ms`):

4.1 `veterinaria-mascota-ms-dev.yml`

```
server:
  port: 8080 # fijo: ya no usa ${PORT:0}; 2da instancia via --server.port
spring:
  datasource:
    url: jdbc:postgresql://localhost:15432/veterinaria_mascota_db
    username: veterinaria
    password: veterinaria # valor literal (hardcodeado), aceptable en DEV local
  jpa:
    hibernate:
      ddl-auto: validate
      show-sql: true
  springdoc:
    swagger-ui:
      path: /swagger-ui.html # Swagger habilitado en DEV
```

4.2 `veterinaria-mascota-ms-prod.yml`

```
server:
  port: 8080
spring:
  datasource:
    url: jdbc:postgresql://${DB_HOST}:${DB_PORT}/${DB_NAME}
    username: ${DB_USER}
    password: ${DB_PASS} # ya NO hardcodeado: viene de variable de entorno
  jpa:
    hibernate:
      ddl-auto: validate
      show-sql: false
  springdoc:
    swagger-ui:
      enabled: false # Swagger deshabilitado en PROD
```

Valor identificado (tarea 5 de la guía): en `veterinaria-mascota-ms-dev.yml` la credencial `spring.datasource.password` está literalmente escrita como `veterinaria`. En `veterinaria-mascota-ms-prod.yml` ese mismo campo pasó a ser `${DB_PASS}`: ya no hay ningún secreto hardcodeado en el archivo versionado, el valor real se inyecta por variable de entorno al desplegar. Antes de S2 solo existía un `application-dev.yml` local con la contraseña escrita en texto plano dentro del propio repositorio del microservicio; ahora esa decisión (hardcodear o parametrizar) se toma por archivo de perfil en `config-repo`, no en el código del servicio.

5. Migración de `veterinaria-mascota-ms` a Config Client

5.1 Cambios en `pom.xml`

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-config</artifactId>
</dependency>
<!-- + mismo spring-cloud.version / dependencyManagement que 3.1 -->
```

5.2 `application.yml` : antes (S1) vs ahora (S2)

Antes — `application.yml` + `application-dev.yml` + `application.properties` (3 archivos, config local) Ahora — `application.yml` único (config externa)

```
spring:
  application:
    name: veterinaria-mascota-ms
  profiles:
    active: dev
# + application-dev.yml con datasource,
# flyway, jpa, swagger, actuator...
# + application.properties duplicando
# el nombre de la app
```

```
spring:
  application:
    name: veterinaria-mascota-ms
  profiles:
    active: dev
  config:
    import: "optional:configserver:
      ${CONFIG_SERVER_URL:
        http://localhost:18888}"
```

`application-dev.yml` y `application.properties` se eliminaron del microservicio: todo su contenido vive ahora en `config-repo`. El prefijo `optional:` evita que el arranque falle si `veterinaria-config` está apagado (aunque el microservicio de todos modos necesita PostgreSQL real para funcionar).

6. Actividad autónoma: replicación del patrón en citas-ms

Siguiendo la sección 4 de la guía ("CREA"), se construyó un segundo microservicio completo —**citas-ms**, gestión de citas veterinarias— ya integrado como Config Client desde su primer commit, replicando exactamente la arquitectura en capas de S1 (Filter → Controller → DTO → Service → Mapper → Entity → Repository) sobre un dominio nuevo.

6.1 Dominio

Entidad	Campos	Notas
Cita	id , mascotaId , motivo , fechaHora , estado	mascotaId es un Long simple (sin FK real): cada microservicio tiene su propia base de datos, no hay join entre citas_db y mascota_db a este nivel — ese acoplamiento se resolverá vía llamadas HTTP entre servicios en sesiones posteriores.

Base de datos propia (veterinaria_citas_db , puerto 15433) y puerto de aplicación propio (8095) para no colisionar con veterinaria-mascota-ms (8080/8081). CRUD completo expuesto en /api/v1/citas .

6.2 citas-ms-dev.yml / citas-ms-prod.yml

Mismo patrón de contraste dev/prod que la sección 4: en dev, spring.datasource.password: citas literal; en prod, \${DB_PASS} por variable de entorno; Swagger habilitado solo en dev; show-sql solo en dev.

7. Evidencia de ejecución real

Se levantó la pila completa: dos PostgreSQL (uno por microservicio), el Config Server, y ambos microservicios como clientes —incluyendo una **segunda instancia** de `veterinaria-mascota-ms` en el puerto 8081 para confirmar que el escalamiento horizontal de S1 sigue funcionando con configuración centralizada.

```
$ cd infra/veterinaria-config && ./mvnw spring-boot:run # :18888
$ cd services/veterinaria-mascota-ms && docker compose -f compose-dev.yml up -d
$ ./mvnw spring-boot:run # :8080
$ java -jar target/veterinaria-mascota-ms-*.jar --server.port=8081 # 2da instancia
$ cd services/citas-ms && docker compose -f compose-dev.yml up -d
$ ./mvnw spring-boot:run # :8095
```

7.1 Config Server operativo (Bloque 1 de la guía)

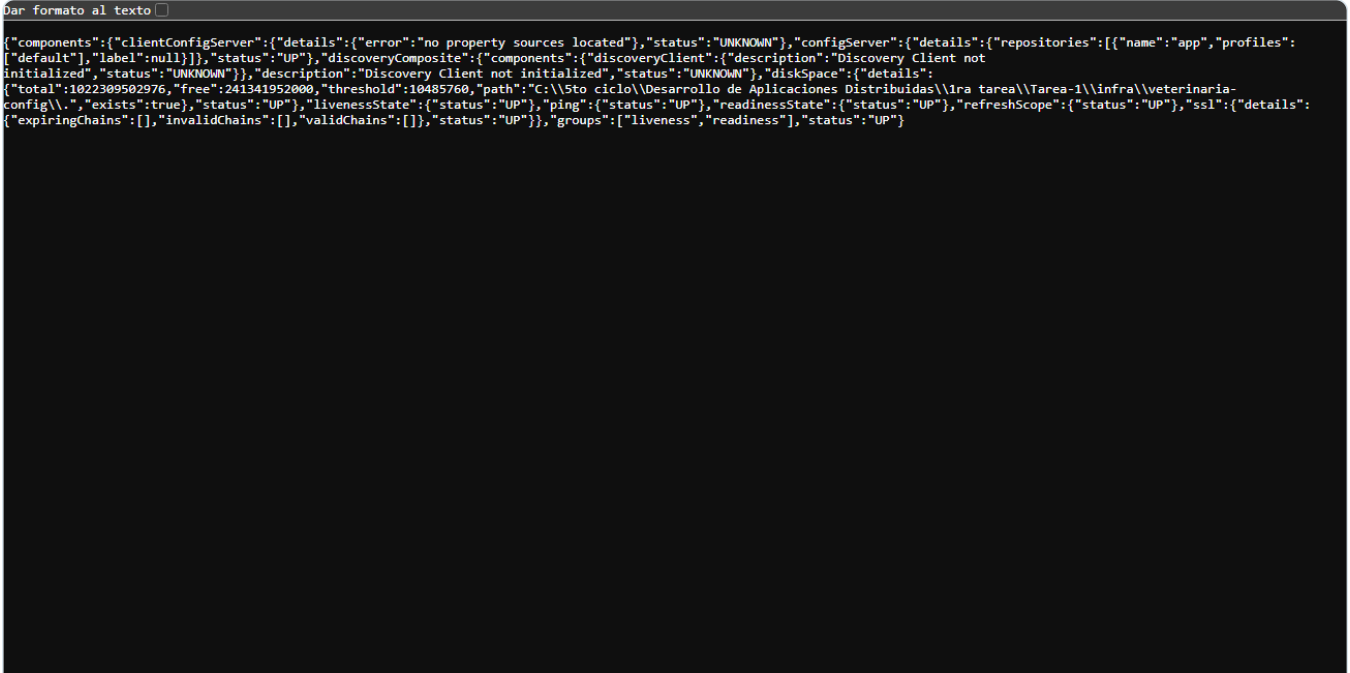


Figura 1 — GET /actuator/health de veterinaria-config en el puerto 18888: configServer.status: UP , repositorio config-repo localizado.

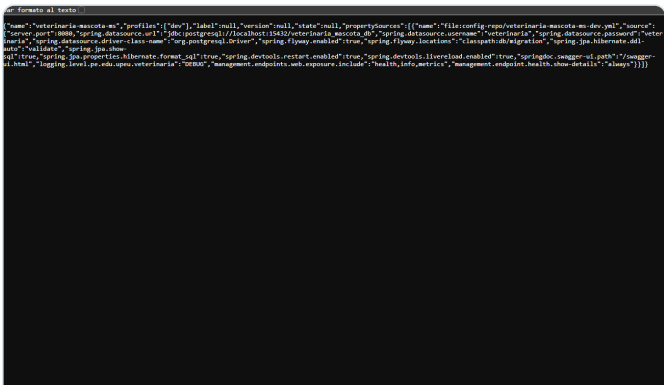


Figura 2 — GET /veterinaria-mascota-ms/dev : propertySources apunta a file:config-repo/veterinaria-mascota-ms-dev.yml , con server.port , spring.datasource.* , etc.

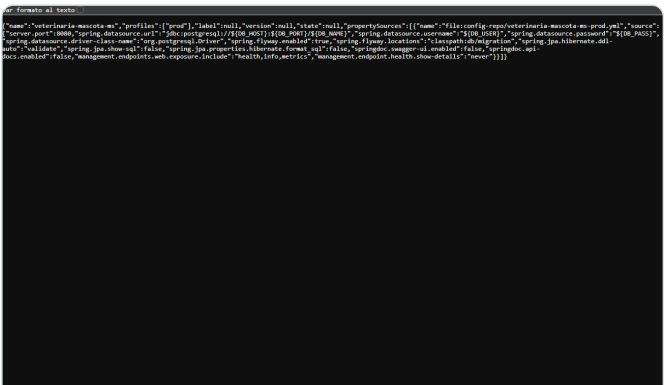


Figura 3 — GET /veterinaria-mascota-ms/prod : mismo esquema, ahora resuelto contra veterinaria-mascota-ms-prod.yml (valores de PROD, show-sql:false , Swagger deshabilitado).

7.2 Configuración externa de citas-ms (Bloque 2 de la guía)

Figura 4 — GET /citas-ms/dev : nuevo archivo `citas-ms-dev.yml` servido correctamente por el mismo Config Server, sin tocar código de `citas-ms`.

Figura 5 — GET /citas-ms/prod : perfil de producción de citas-ms , con credenciales parametrizadas.

Figura 6 — Consulta con nombre incorrecto `GET /mascota/dev` (en vez de `veterinaria-mascota-ms`): responde `200 OK` pero con `"propertySources": []` — el Config Server no falla, simplemente no encuentra coincidencia porque el nombre no es exacto (ver sección 9, `error/hallazgo`).

Figura 7 — `veterinaria-mascota-ms` (puerto 8080): `clientConfigServer.status: UP`, con `propertySources: ["configserver:file:config-repo/veterinaria-mascota-ms-dev.yml", ...]` — prueba directa de que este arranque leyó configuración remota, no un archivo local.

Figura 8— citas-ms (puerto 8095): mismo `clientConfigServer` apuntando a `citas-ms-dev.yml`, base de datos propia UP.

Dar formato al texto ☐

```
[{"id":1,"mascotaId":1,"motivo":"Control anual de vacunas","fechaHora":"2026-09-02T10:30:00","estado":"PENDIENTE"}]
```

Figura 9 — GET /api/v1/citas respondiendo 200 OK con una cita creada por POST durante esta verificación: el CRUD de citas-ms funciona igual que veterinaria-mascota-ms en S1, ahora con configuración 100% externa.

7.4 Segunda instancia de veterinaria-mascota-ms (escalamiento + config centralizada)

```
$ curl http://localhost:8081/actuator/health | jq '.components.clientConfigServer.details.propertySources'
["configserver:file:config-repo/veterinaria-mascota-ms-dev.yml","configClient"]
-- misma fuente que la instancia de 8080: ambas leen la config centralizada, sin archivo local --
```

8. Comparación DEV vs PROD y comprensión (Bloque 4 de la guía)

Propiedad	*-dev.yml	*-prod.yml	Por qué difiere
<code>spring.datasource.password</code>	<code>veterinaria / citas</code> (literal)	<code>\${DB_PASS}</code> (env var)	En DEV la BD es local y descartable; en PROD el secreto no puede vivir en un archivo versionado en Git.
<code>spring.datasource.url</code>	<code>jdbc:postgresql://localhost:PUERTO/...</code>	<code>jdbc:postgresql://\${DB_HOST}:\${DB_PORT}/\${DB_NAME}</code>	PROD no conoce de antemano el host de la BD (puede ser un contenedor, un servicio gestionado, etc.).
<code>spring.jpa.show-sql</code>	<code>true</code>	<code>false</code>	Ver SQL ayuda a depurar en DEV; en PROD es ruido y filtra estructura interna en logs.
<code>springdoc.swagger-ui</code>	<code>habilitado (path: /swagger-ui.html)</code>	<code>enabled: false</code>	Exponer la documentación interactiva de la API en producción es superficie de ataque innecesaria.
<code>management.endpoint.health.show-details</code>	<code>always</code>	<code>never</code>	En DEV el detalle ayuda a diagnosticar; en PROD revelar detalles de infraestructura en <code>/actuator/health</code> es una fuga de información.

Cómo Config Server separa código y configuración (explicación propia)

Antes de S2, el "código" y la "configuración de ambiente" de `veterinaria-mascota-ms` vivían en el mismo artefacto: `application-dev.yml` estaba empaquetado dentro del `.jar`, así que cambiar una URL de base de datos exigía recompilar y volver a desplegar. Con Config Server, el `.jar` ya no contiene ningún valor de ambiente: solo sabe su propio nombre (`spring.application.name`) y dónde preguntar (`CONFIG_SERVER_URL`). Los valores reales —URLs, credenciales, flags de Swagger— se movieron a `config-repo`, un lugar versionado por separado. Esto significa que el mismo binario compilado una sola vez puede correr en DEV o en PROD sin cambiar una línea de código: solo cambia qué perfil activa y contra qué Config Server pregunta. También significa que un cambio de configuración (por ejemplo, bajar el nivel de log) no requiere una nueva build, solo editar el archivo en `config-repo` y reiniciar el microservicio para que vuelva a preguntar.

9. Error / hallazgo técnico real

Puerto ya en uso al levantar `citas-ms` . El plan original asignaba el puerto `8090` a `citas-ms-dev.yml` . Al arrancar, Spring Boot falló con:

```
Web server failed to start. Port 8090 was already in use.
```

Diagnóstico: `netstat -ano | grep :8090` mostró un proceso escuchando en ese puerto; `tasklist /FI "PID eq <pid>"` reveló que era `WsToastNotification.exe` , un proceso de Windows ajeno por completo al proyecto —no otra instancia de Java. Solución: se cambió `server.port` a `8095` en `citas-ms-dev.yml` y `citas-ms-prod.yml` dentro de `config-repo` , sin tocar código ni recompilar el microservicio, y al reiniciar `citas-ms` tomó el nuevo puerto automáticamente desde el Config Server. Esto ilustra en la práctica la ventaja central del patrón: el ajuste se hizo en un archivo de configuración centralizado, no en el `.jar` .

9.1 Diagnóstico de perfil no encontrado (tabla de la guía, verificada)

Problema	Causa	Cómo se comprobó aquí
Respuesta 200 con <code> propertySources: [] </code>	Nombre de aplicación no coincide	Sección 7.2, Figura 6: <code> /mascota/dev </code> en vez de <code> /veterinaria-mascota-ms/dev </code>
Microservicio arranca con valores vacíos	Falta <code> spring.config.import </code>	Verificado por diseño: ambos microservicios lo declaran en <code> application.yml </code>
Microservicio no arranca	Config Server apagado	El prefijo <code> optional: </code> evita el fallo duro; se verificó levantando Config Server primero en todos los arranques

10. Reflexión técnica

Con dos microservicios ya es visible el problema que resuelve Config Server: antes de S2, cada uno cargaba su propia copia de `application-dev.yml`, y cualquier convención nueva (por ejemplo, agregar `management.endpoint.health.show-details`) había que replicarla manualmente en cada proyecto, con riesgo de que un microservicio quedara desactualizado o con un valor distinto por descuido. Con `config-repo` centralizado, esa misma convención se define una sola vez por ambiente y todos los microservicios que la necesiten la heredan al consultar su propio archivo. A la escala de dos servicios el ahorro es modesto, pero es lineal en el número de microservicios y de instancias: si en sesiones futuras (S3 Eureka, S4 Gateway) el sistema crece a cinco o diez microservicios con varias instancias cada uno, la alternativa —archivos de configuración locales por instancia— se vuelve inauditable: no hay forma rápida de responder "¿qué URL de base de datos está usando ahora mismo la instancia 3 de auth-ms en producción?" sin entrar a cada máquina. Con Config Server, esa respuesta es una sola petición HTTP al mismo lugar para cualquier servicio, cualquier ambiente. El costo es que el Config Server se vuelve un punto único de arranque: por eso el prefijo `optional:` y por eso, en producción real, este patrón normalmente se combina con caché local y alta disponibilidad del propio Config Server (no cubierto en esta sesión).

11. Verificación contra los criterios de aceptación (4.4)

- ✓ `veterinaria-config` responde en DEV con evidencia real (sección 7.1).
- ✓ `citas-ms-dev.yml` y `citas-ms-prod.yml` existen en `config-repo` con valores propios, distintos de los de `veterinaria-mascota-ms` (puerto 8095, BD `veterinaria_citas_db`, usuario `citas`).
- ✓ Microservicio replicado (`citas-ms`) arranca leyendo Config Server, no archivos locales — confirmado por `clientConfigServer.propertySources` en su `/actuator/health` (Figura 8).
- ✓ Comparación DEV/PROD identifica un valor real, no genérico: `spring.datasource.password` hardcodeado en dev vs. `${DB_PASS}` en prod (sección 8).
- △ Producción local con Docker (3.11–3.14 de la guía): **no ejecutada** — explícitamente opcional y fuera del alcance evaluado (guía: "el alcance evaluado termina en 3.10").
- △ Formato individual exacto de la sección 4.3 (PDF nombrado `S02_Equipo##_ApellidoNombre.pdf`, con capturas de **reloj del sistema y usuario/foto de perfil visibles sin recortar**): este informe cubre todo el contenido técnico exigido, pero esas capturas de escritorio con reloj y perfil de Windows/VS Code/navegador solo pueden tomarse en tu propia máquina con una herramienta de captura real (p. ej. `Win+Shift+S`); no es algo que un navegador headless pueda producir de forma honesta. Ver nota al final.
- ✓ Error/hallazgo técnico real incluido, diagnosticado con evidencia de comandos (sección 9).
- ✓ Reflexión técnica incluida (sección 10).

12. Respuestas a las preguntas de sustentación (4.5)

1. ¿Qué problema resuelve la configuración centralizada?

Evita que cada microservicio guarde su propia copia de la configuración de ambiente dentro de su artefacto compilado. Sin ella, cambiar un valor (una URL, un flag) exige recompilar y redespargar cada instancia de cada servicio afectado, y no existe un lugar único para auditar qué configuración corre realmente en cada ambiente.

2. ¿Qué diferencia hay entre Config Server y config-repo?

`veterinaria-config` (Config Server) es la aplicación Spring Boot que expone el protocolo HTTP `/app/{perfil}`; no contiene ningún valor de configuración por sí misma. `config-repo` es simplemente una carpeta con archivos YAML versionables (en este caso local, con perfil `native`; en un entorno real normalmente sería un repositorio Git aparte) que el Config Server lee y sirve tal cual. El servidor es el mecanismo de entrega; el repo es la fuente de verdad de los datos.

3. ¿Cómo se forma la URL `/app/{application}/{profile}` ?

`{application}` es exactamente el valor de `spring.application.name` del microservicio consultante (o consultado manualmente), y `{profile}` es el perfil activo (`dev` , `prod`). El Config Server concatena ambos con un guion y agrega `.yaml` para buscar el archivo `{application}-{profile}.yaml` dentro de `search-locations` . Ej.: `/citas-ms/dev` → `citas-ms-dev.yaml` .

4. ¿Qué debe coincidir exactamente entre `spring.application.name` y archivos de `config-repo` ?

El nombre completo, carácter por carácter, incluyendo guiones. `veterinaria-mascota-ms` solo encuentra `veterinaria-mascota-ms-dev.yaml` ; un archivo llamado `mascota-dev.yaml` o `veterinariaMascotaMs-dev.yaml` nunca se resolvería, aunque el contenido fuera idéntico.

5. ¿Cómo diagnosticas un perfil no encontrado?

Primero se consulta directamente al Config Server con `curl http://localhost:18888/{nombre}/{perfil}` : si devuelve `propertySources: []` , el nombre o perfil no coincide con ningún archivo (como se reprodujo deliberadamente en la Figura 6). Si el Config Server ni siquiera responde, el problema es anterior (servidor apagado o `search-locations` mal configurado). Si el Config Server sí devuelve valores pero el microservicio sigue arrancando con propiedades vacías, el problema está en el cliente: falta `spring.config.import` o apunta a una URL distinta.

13. Conclusiones

`veterinaria-config` queda operativo en DEV, sirviendo por HTTP la configuración de ambos microservicios del sistema. `veterinaria-mascota-ms` se migró completamente desde configuración local (S1) a Config Client (S2) sin perder ninguna funcionalidad verificada previamente —CRUD, Swagger, Actuator, escalamiento horizontal—, y `citas-ms` demuestra que el mismo patrón se replica en un microservicio nuevo sin fricción. El único punto realmente pendiente es de formato de entrega individual, no de arquitectura: las capturas con reloj/usuario de la sección 4.3 de la guía requieren una herramienta de captura de pantalla real sobre tu propio escritorio, algo que este informe no puede fabricar de forma honesta.

Informe generado a partir de la inspección del código fuente en `Tarea-1` y de una ejecución real de la pila completa (2 PostgreSQL + Config Server + 3 instancias de microservicio) el 27 de agosto de 2026.

Guía de referencia: 262dist.github.io/pagatu/sesiones/S02_Configuracion_Centralizada_Ambientes/